Daily Sales Loader

Code:

```
import os
import logging
from datetime import datetime, timedelta

import pandas as pd

from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.snowflake.hooks.snowflake import SnowflakeHook
from airflow.exceptions import AirflowException


logger = logging.getLogger(__name__)


def create_table():

    try:

        hook = SnowflakeHook(
            snowflake_conn_id='snowflake_default'
        )

        create_table_sql = """
        CREATE TABLE IF NOT EXISTS RAW_SALES (
            order_id INT,
            product VARCHAR,
            quantity INT,
            unit_price FLOAT,
            sale_date DATE,
            region VARCHAR,
            loaded_at TIMESTAMP
        );
        """

        hook.run(create_table_sql)

        logger.info("Table RAW_SALES ready.")

    except Exception as e:

        logger.error(
```

```
            f"Error while creating RAW_SALES table: {str(e)}"
        )

        raise AirflowException(
            f"create_table task failed: {str(e)}"
        )


def clear_todays_data():

    try:

        hook = SnowflakeHook(
            snowflake_conn_id='snowflake_default'
        )

        count_query = """
        SELECT COUNT(*)
        FROM RAW_SALES
        WHERE loaded_at::DATE = CURRENT_DATE();
        """

        deleted_rows = hook.get_first(count_query)[0]

        delete_query = """
        DELETE FROM RAW_SALES
        WHERE loaded_at::DATE = CURRENT_DATE();
        """

        hook.run(delete_query)

        logger.info(
            f"Deleted {deleted_rows} rows from RAW_SALES."
        )

    except Exception as e:

        logger.error(
            f"Error while clearing today's data: {str(e)}"
        )

        raise AirflowException(
            f"clear_todays_data task failed: {str(e)}"
        )



def load_csv_to_snowflake():
    try:
```

```
        file_path = "data/sales_data.csv"

        if not os.path.exists(file_path):
            raise AirflowException(
                f"CSV file not found: {file_path}"
            )

        df = pd.read_csv(file_path)

        hook = SnowflakeHook(
            snowflake_conn_id="snowflake_default"
        )

        for _, row in df.iterrows():

            product = str(row['product']).replace("'", "''")
            region = str(row['region']).replace("'", "''")

            insert_sql = f"""
            INSERT INTO RAW_SALES (
                order_id,
                product,
                quantity,
                unit_price,
                sale_date,
                region,
                loaded_at
            )
            VALUES (
                {row['order_id']},
                '{product}',
                {row['quantity']},
                {row['unit_price']},
                '{row['sale_date']}',
                '{region}',
                CURRENT_TIMESTAMP()
            )
            """

            try:

                hook.run(insert_sql)

            except Exception as e:

                logger.error(
                    f"Failed inserting order_id "
                    f"{row['order_id']}: {str(e)}"
```

```
                )

                raise AirflowException(
                    "CSV load failed."
                )

        logger.info(
            f"Loaded {len(df)} rows into RAW_SALES."
        )
    except Exception as e:

        logger.error(
            f"Error in load_csv_to_snowflake: {str(e)}"
        )

        raise AirflowException(
            f"load_csv_to_snowflake failed: {str(e)}"
        )

def transform_in_snowflake():

    try:

        hook = SnowflakeHook(
            snowflake_conn_id='snowflake_default'
        )

        # Create summary table
        create_summary_table = """
        CREATE TABLE IF NOT EXISTS SALES_SUMMARY (
            region VARCHAR,
            total_orders INT,
            total_revenue FLOAT,
            avg_order_value FLOAT,
            summary_date DATE
        );
        """

        hook.run(create_summary_table)

        logger.info("SALES_SUMMARY table ready.")

        # Idempotency delete
        delete_query = """
        DELETE FROM SALES_SUMMARY
        WHERE summary_date = CURRENT_DATE();
        """
```

```
        hook.run(delete_query)

        logger.info(
            "Deleted today's existing SALES_SUMMARY rows."
        )

        # Insert transformed data
        insert_query = """
        INSERT INTO SALES_SUMMARY (
            region,
            total_orders,
            total_revenue,
            avg_order_value,
            summary_date
        )
        SELECT
            region,
            COUNT(*),
            SUM(quantity * unit_price),
            AVG(quantity * unit_price),
            CURRENT_DATE()
        FROM RAW_SALES
        WHERE loaded_at::DATE = CURRENT_DATE()
        GROUP BY region;
        """

        hook.run(insert_query)

        count_query = """
        SELECT COUNT(*)
        FROM SALES_SUMMARY
        WHERE summary_date = CURRENT_DATE();
        """

        region_count = hook.get_first(count_query)[0]

        logger.info(
            f"{region_count} region summary rows written."
        )

    except Exception as e:

        logger.error(
            f"Error in transform_in_snowflake: {str(e)}"
        )

        raise AirflowException(
            f"transform_in_snowflake failed: {str(e)}"
```

```
        )


def quality_check():

    try:

        hook = SnowflakeHook(
            snowflake_conn_id='snowflake_default'
        )

        row_count_query = """
        SELECT COUNT(*)
        FROM RAW_SALES
        WHERE loaded_at::DATE = CURRENT_DATE();
        """

        row_count = hook.get_first(
            row_count_query
        )[0]

        if row_count == 0:

            logger.error(
                "FAIL: No rows found in RAW_SALES."
            )

            raise AirflowException(
                "Quality Check Failed: No rows loaded."
            )

        logger.info(
            f"PASS: Row count check passed. Rows = {row_count}"
        )

        null_check_query = """
        SELECT COUNT(*)
        FROM RAW_SALES
        WHERE product IS NULL
           OR unit_price IS NULL;
        """

        null_count = hook.get_first(
            null_check_query
        )[0]

        if null_count > 0:

            logger.warning(
```

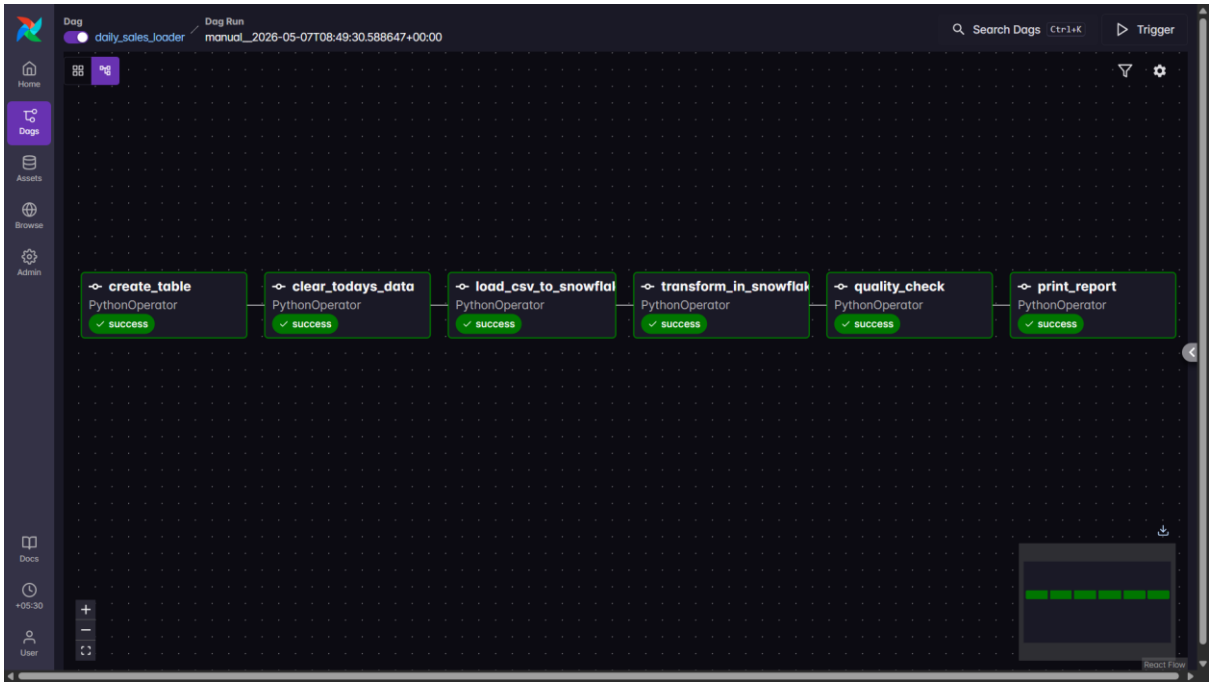
```
                f"WARNING: Found {null_count} NULL rows."
            )

        else:

            logger.info(
                "PASS: NULL check passed."
            )

        negative_price_query = """
        SELECT COUNT(*)
        FROM RAW_SALES
        WHERE unit_price < 0;
        """

        negative_count = hook.get_first(
            negative_price_query
        )[0]

        if negative_count > 0:

            logger.error(
                f"FAIL: Found {negative_count} negative prices."
            )

            raise AirflowException(
                "Quality Check Failed: Negative prices found."
            )

        logger.info(
            "PASS: Negative price check passed."
        )

        logger.info(
            "All quality checks completed successfully."
        )

    except Exception as e:

        logger.error(
            f"Error in quality_check: {str(e)}"
        )

        raise AirflowException(
            f"quality_check failed: {str(e)}"
        )


def print_report():
```

```
try:

    hook = SnowflakeHook(
        snowflake_conn_id='snowflake_default'
    )

    report_query = """
    SELECT
        region,
        total_orders,
        total_revenue,
        avg_order_value
    FROM SALES_SUMMARY
    WHERE summary_date = CURRENT_DATE();
    """

    report_data = hook.get_records(
        report_query
    )

    logger.info(
        "========== SALES REPORT =========="
    )

    grand_total = 0

    for row in report_data:

        region = row[0]
        total_orders = row[1]
        total_revenue = row[2]
        avg_order_value = row[3]

        grand_total += total_revenue

        logger.info(
            f"""
            Region: {region}
            Total Orders: {total_orders}
            Total Revenue: {total_revenue}
            Average Order Value: {avg_order_value}
            """
        )

    logger.info(
        f"GRAND TOTAL REVENUE: {grand_total}"
    )
```

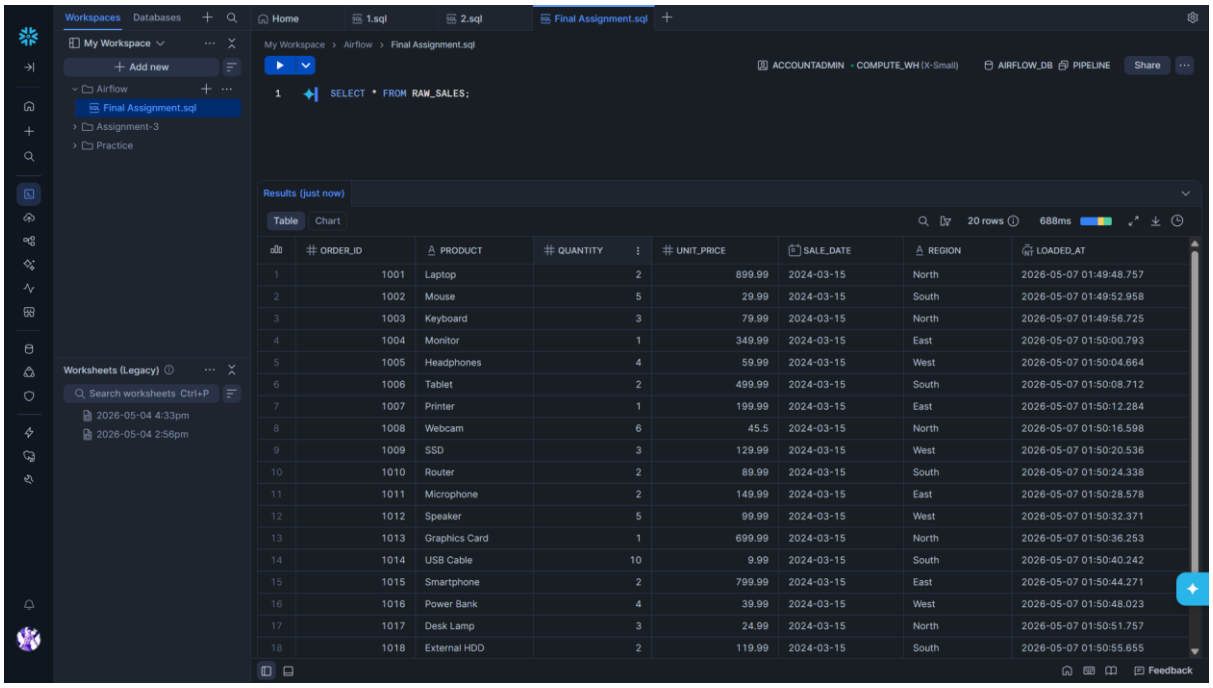
```
        logger.info(
            "========== END REPORT =========="
        )

    except Exception as e:

        logger.error(
            f"Error in print_report: {str(e)}"
        )

        raise AirflowException(
            f"print_report failed: {str(e)}"
        )

default_args = {
    'owner': 'airflow',
    'retries': 2,
    'retry_delay': timedelta(minutes=5)
}


with DAG(
    dag_id='daily_sales_loader',
    schedule='@daily',
    start_date=datetime(2023, 1, 1),
    catchup=False,
    default_args=default_args
) as dag:

    create_table_task = PythonOperator(
        task_id='create_table',
        python_callable=create_table
    )

    clear_todays_data_task = PythonOperator(
        task_id='clear_todays_data',
        python_callable=clear_todays_data
    )

    load_csv_task = PythonOperator(
        task_id='load_csv_to_snowflake',
        python_callable=load_csv_to_snowflake
    )

    transform_task = PythonOperator(
        task_id='transform_in_snowflake',
        python_callable=transform_in_snowflake
    )
```

```
    quality_check_task = PythonOperator(
        task_id='quality_check',
        python_callable=quality_check
    )

    print_report_task = PythonOperator(
        task_id='print_report',
        python_callable=print_report
    )

    create_table_task >> clear_todays_data_task >> load_csv_task >>
transform_task >> quality_check_task >> print_report_task
```

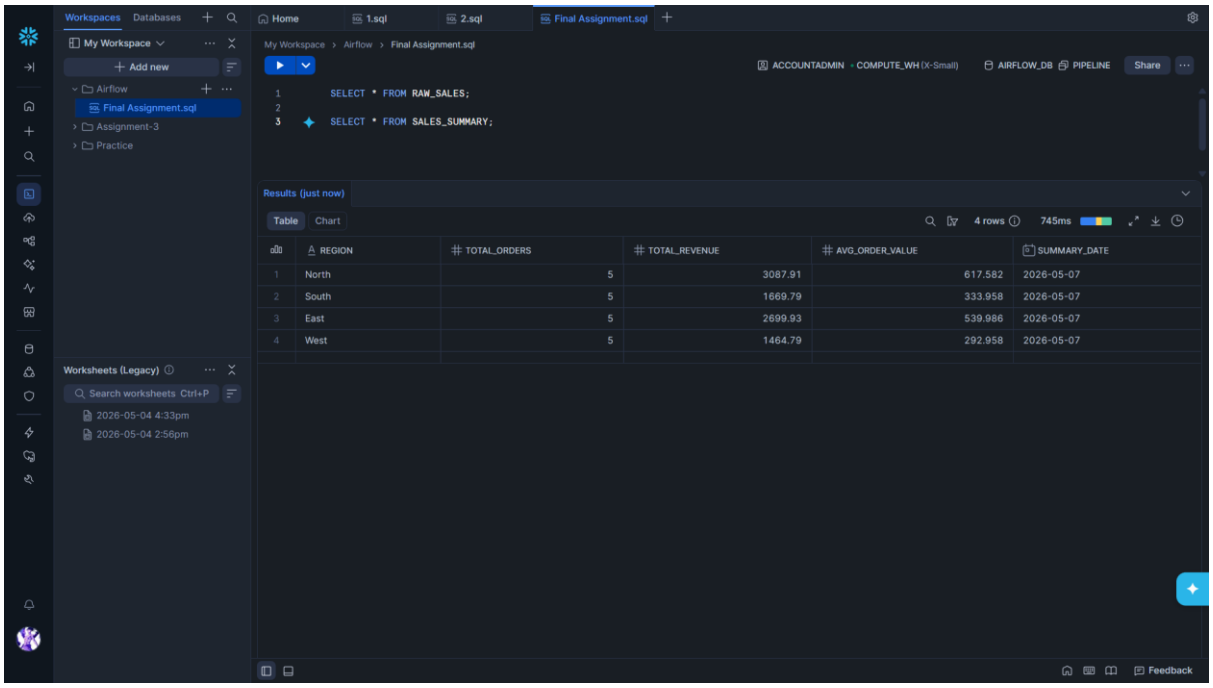
1. Graph View — all 6 tasks GREEN in correct sequence



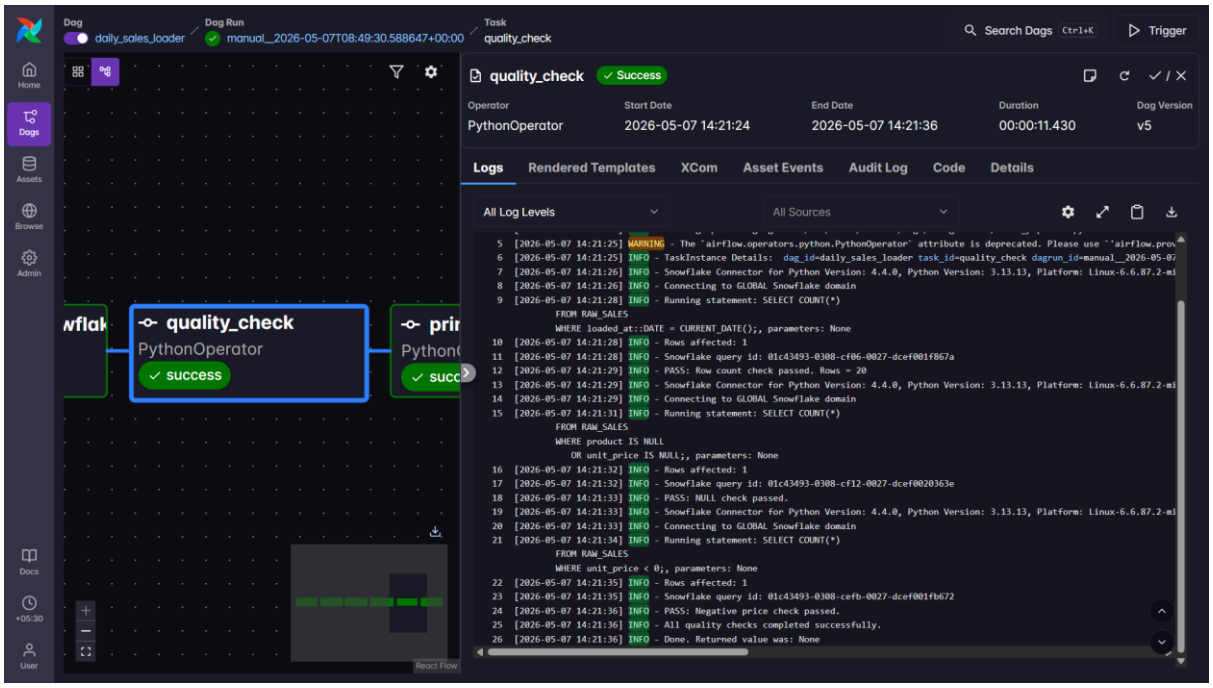
2. Snowflake Worksheet — SELECT * FROM RAW_SALES showing loaded rows with loaded_at column



3. Snowflake Worksheet — SELECT * FROM SALES_SUMMARY showing grouped revenue per region



4. Task Log of quality_check — PASS printed for all 3 checks with the actual values shown



5. Task Log of print_report — formatted region report with grand total revenue